Relazione di Laboratorio

Analisi empirica degli algoritmi di ordinamento

AZIADZRO Komla Achile
Matricola: 170110
Email: 170110@spes.uniud.it

Tommaso Persello
Matricola: 163673
Email: 163673@spes.uniud.it

Corso di Algoritmi e Strutture dei Dati e Laboratorio
Anno Accademico 2024/2025

Indice

1. **Introduzione**
    - Obiettivo del progetto
    - Riepilogo degli algoritmi scelti
2. **Algoritmi implementati**
    - QuickSort
    - CountingSort
    - QuickSort 3-way
    - MergeSort
3. **Metodo sperimentale**
    - Stima della risoluzione del clock e calcolo di $T_{\min}$
    - Procedura di misura
    - Scelta dei valori di n e m: progressione geometrica
4. **Risultati**
    - Confronto rispetto a n (m fisso)
    - Confronto rispetto a m (n fisso)
    - Comportamento per input di grandi dimensioni
    - Conclusioni intermedie
5. **Conclusioni**
    - Riepilogo dei risultati
    - Scelte progettuali e miglioramenti possibili
    - Giustificazione della scelta di MergeSort come quarto algoritmo
    - Conclusione finale

1 Introduzione

Obiettivo del progetto

Il progetto proposto nell'ambito del corso di *Algoritmi e Strutture dei Dati e Laboratorio* ha come obiettivo principale l'analisi empirica delle prestazioni di diversi algoritmi di ordinamento su array di numeri interi. In particolare, si intende:

- progettare e implementare versioni corrette ed efficienti degli algoritmi richiesti;
- analizzarne il comportamento al variare delle dimensioni dell'input (n) e del dominio dei valori (m);
- confrontare i risultati ottenuti con le complessità teoriche studiate a lezione;
- documentare sperimentalmente le differenze tra algoritmi comparativi e non comparativi;
- fornire una discussione critica sui risultati ottenuti, evidenziando punti di forza e di debolezza di ciascun approccio.

L'intero lavoro si inserisce in una prospettiva didattica volta non solo a consolidare le nozioni teoriche apprese (come le analisi nel caso medio, le proprietà di stabilità o la dipendenza dal dominio dei dati), ma anche a sviluppare competenze di tipo ingegneristico: progettazione modulare, accuratezza nella misurazione dei tempi, rigore sperimentale e capacità di interpretazione dei dati.

Attraverso questo progetto, si intende quindi non solo dimostrare la correttezza formale degli algoritmi implementati, ma anche investigare empiricamente la loro efficienza e le condizioni in cui ciascun algoritmo risulta preferibile rispetto agli altri.

Riepilogo degli algoritmi scelti

Nel rispetto delle indicazioni fornite dal docente, sono stati implementati e testati i seguenti quattro algoritmi:

- **QuickSort**: algoritmo ricorsivo, comparativo, basato sulla scelta di un pivot e la partizione dell'array in sottosequenze da ordinare separatamente. È noto per la sua velocità nel caso medio, ma può degenerare nel caso peggiore.
- **CountingSort**: algoritmo non comparativo, che sfrutta il conteggio delle occorrenze dei valori nell'array. È estremamente efficiente se il dominio dei valori interi è limitato (m piccolo), ma diventa inefficiente per m grandi.
- **QuickSort 3-way**: una variante di QuickSort ottimizzata per array contenenti molti duplicati. Utilizza una partizione in tre sezioni (minori, uguali, maggiori) che riduce il numero di confronti ridondanti.
- **MergeSort** (scelto liberamente come quarto algoritmo): algoritmo stabile, ricorsivo, divide et impera, garantisce una complessità $O(n \log n)$ anche nel caso peggiore. È adatto a contesti in cui si richiede la preservazione dell'ordine relativo tra elementi uguali.

La selezione di questi algoritmi è motivata dalla volontà di coprire scenari diversi: algoritmi stabili e non stabili, comparativi e non comparativi, sensibili o meno alla presenza di duplicati, in-place o meno. Tale varietà permette un confronto più ampio e significativo dal punto di vista sperimentale.

2 Algoritmi implementati

Nel corso del progetto sono stati implementati quattro algoritmi di ordinamento, scelti per rappresentare in maniera equilibrata le principali classi analizzate nel corso: algoritmi **comparativi** e **non comparativi**, **in-place** e **non in-place**, **stabili** e **non stabili**.

Ogni algoritmo è stato studiato sia dal punto di vista **teorico** (complessità, stabilità, sensibilità all'input), sia attraverso l'**implementazione pratica**, nel linguaggio Python.

2.1 QuickSort

QuickSort è un algoritmo ricorsivo basato sul paradigma *divide et impera*. Dato un array A, si sceglie un elemento come pivot e si partiziona l'array in due sottoarray:

- gli elementi $<$ pivot,
- e quelli $>$ pivot.

Si procede ricorsivamente su ciascun sottoarray.

Complessità asintotica

- **Caso migliore:**

$$T(n) = O(n \log n)$$

  Si verifica quando il pivot divide l'array in due metà quasi uguali.

- **Caso medio:**

$$T(n) = \Theta(n \log n)$$

  Dimostrato nel CLRS (cap. 7), assumendo distribuzione uniforme dei pivot.

- **Caso peggiore:**

$$T(n) = O(n^2)$$

  Quando l'array è già ordinato o tutti gli elementi sono uguali, e il pivot scelto è sempre l'estremo.

Note teoriche

- Non è stabile.
- È in-place (richiede solo $O(\log n)$ spazio per la ricorsione).
- L'efficienza dipende criticamente dalla strategia di scelta del pivot.

2.2 CountingSort

CountingSort è un algoritmo **non comparativo**, che si basa sul conteggio delle occorrenze dei valori presenti nell'array. Presuppone che tutti i valori siano interi positivi entro un intervallo $[0, m]$.

L'algoritmo costruisce un array ausiliario C di lunghezza $m+1$, in cui $C[i]$ rappresenta il numero di occorrenze del valore i. A partire da questa struttura, è possibile ricostruire l'array ordinato.

Complessità asintotica

- **Qualsiasi caso:**

$$T(n, m) = \Theta(n + m)$$

  Lo spazio richiesto è anch'esso $O(n + m)$.

Note teoriche

- Algoritmo **stabile**, spesso usato come base per RadixSort.
- Molto efficiente solo quando m non è molto maggiore di n.
- Non è comparativo, quindi **non soggetto al limite inferiore** $\Omega(n \log n)$.

Riferimenti

- Descritto nel CLRS cap. 8.2.
- Appare tra gli algoritmi lineari discussi nel secondo semestre del corso ASD.

2.3 QuickSort 3-way

Questa variante di QuickSort, descritta anche nel testo di Sedgewick, è progettata per gestire efficientemente array con molti duplicati. Invece di una partizione binaria, effettua una partizione ternaria:

- elementi $<$ pivot
- elementi $=$ pivot
- elementi $>$ pivot

L'idea chiave è che se molti elementi sono uguali al pivot, evitare di ricorsivamente riordinare segmenti ridondanti migliora le prestazioni reali.

Complessità asintotica

- **Caso migliore e medio:**

$$T(n) = O(n \log n)$$

- **Caso peggiore:**

$$T(n) = O(n^2)$$

Tuttavia, nei casi reali con duplicati, si comporta **meglio** di QuickSort standard.

3 Metodo sperimentale

Per valutare empiricamente le prestazioni dei quattro algoritmi implementati, è stato sviluppato un protocollo di misura che mira a ridurre l'errore di campionamento e garantire un confronto omogeneo tra gli algoritmi. La procedura segue le buone pratiche discusse nel corso, in particolare quelle riguardanti la misura dei tempi medi di esecuzione su input pseudo-casuali, la validazione della significatività delle misure, e l'utilizzo di strumenti di timing ad alta risoluzione.

3.1 Stima della risoluzione del clock e calcolo di T_min

Per evitare che le misure siano influenzate dal rumore di fondo del sistema operativo o dalla bassa granularità del timer, è stato calcolato il **tempo minimo misurabile** $T_{\min}$ sulla base della **risoluzione effettiva del clock monotono** del sistema.

Utilizzando il timer `time.perf_counter()` in Python (analogo a `System.nanoTime()` in Java), è stata stimata la risoluzione R tramite un ciclo di confronto tra timestamp consecutivi:

$$R = \min\{t_1 - t_0 : t_1 > t_0\}$$

A partire da R, è stato calcolato il valore di **tempo minimo affidabile di misura** utilizzando la formula teorica vista in laboratorio:

$$T_{\min} = R \cdot \left(\frac{1}{E} + 1\right)$$

dove E rappresenta l'**errore relativo massimo tollerabile**, fissato a 0.001.

Questo valore di $T_{\min}$ ha poi guidato il numero minimo di ripetizioni necessarie in ogni esperimento: ogni algoritmo veniva rieseguito su nuovi array casuali finché il tempo cumulativo superava $T_{\min}$, garantendo così un errore statistico controllato.

3.2 Procedura di misura

Per ciascuna configurazione sperimentale (valore di n o m), la procedura adottata è stata la seguente:

1. Generazione di un array pseudo-casuale di n interi, distribuiti uniformemente nell'intervallo $[1, m]$.

2. Esecuzione dell'algoritmo di ordinamento su una copia dell'array.
3. Misurazione del tempo di esecuzione.
4. Ripetizione del processo fino a raggiungere $T_{\min}$ complessivo.
5. Calcolo del **tempo medio di ordinamento** per singola esecuzione come rapporto tra tempo totale e numero di iterazioni.

La generazione dei dati è stata separata dal codice dell'algoritmo per evitare che il tempo di creazione degli array influenzasse i risultati.

Questa procedura è stata applicata a **ciascuno dei quattro algoritmi**, per ciascun valore di n o m testato.

3.3 Scelta dei valori di n e m: progressione geometrica

Per garantire una distribuzione regolare e significativa dei punti di misura, sono stati utilizzati **100 valori distinti** per ciascun parametro, generati con una **progressione geometrica**, secondo la formula:

$$x_i = \left\lfloor A \cdot B^i \right\rfloor, \quad \text{con } i = 0, 1, \ldots, 99$$

dove:

- x_i è il valore di n (o m) da usare al passo i,
- A è un valore iniziale fissato (es. 100 per n, 10 per m),
- B è la base della progressione geometrica, calcolata in modo che:

$$\begin{cases} x_0 = 100,\ x_{99} = 1000000 & \text{per } n \\ x_0 = 10,\ x_{99} = 10000000 & \text{per } m \end{cases}$$

Questa scelta consente di coprire l'intervallo in modo **logaritmico**, e di valutare il comportamento degli algoritmi **sia per piccoli input che per input molto grandi**, senza creare squilibri nel campionamento.

Le due campagne sperimentali principali sono state:

- **Primo esperimento**: variazione di n tra 100 e 1000000 con $m = 100000$ fisso.
- **Secondo esperimento**: variazione di m tra 10 e 10000000 con $n = 10000$ fisso.

4 Risultati

I risultati sperimentali sono stati raccolti in forma tabellare e successivamente visualizzati attraverso **grafici comparativi**, che consentono una lettura immediata del comportamento dei diversi algoritmi in funzione della dimensione dell'input (n) e dell'intervallo dei valori (m). Ogni curva rappresenta il **tempo medio di ordinamento** (in secondi) per un determinato algoritmo, calcolato su almeno 100 esecuzioni indipendenti per ciascun punto.

I grafici sono stati generati utilizzando la libreria `matplotlib` in Python e riportano, per ogni esperimento, due versioni:

- una **in scala lineare**, per evidenziare le differenze nelle costanti di esecuzione;
- una **in scala log-log**, per mettere in luce il comportamento asintotico.

4.1 Confronto rispetto a n (m fisso)

Osservazioni:

- **CountingSort** mostra ottime prestazioni iniziali (per $n < 1000$), ma la curva tende ad appiattirsi e diverge leggermente oltre i 50000 elementi, a causa dell'incidenza crescente della fase di inizializzazione del conteggio.
- **QuickSort** e **MergeSort** hanno un andamento quasi identico in scala log-log, confermando la complessità attesa $O(n \log n)$.
- **QuickSort 3-way** risulta competitivo soprattutto a partire da $n = 10000$, dove la presenza anche minima di duplicati inizia ad avere effetto.

In scala logaritmica, le curve di MergeSort e QuickSort sono quasi parallele e lineari, con pendenza compatibile con $\log(n)$, mentre CountingSort tende ad assumere una pendenza maggiore.

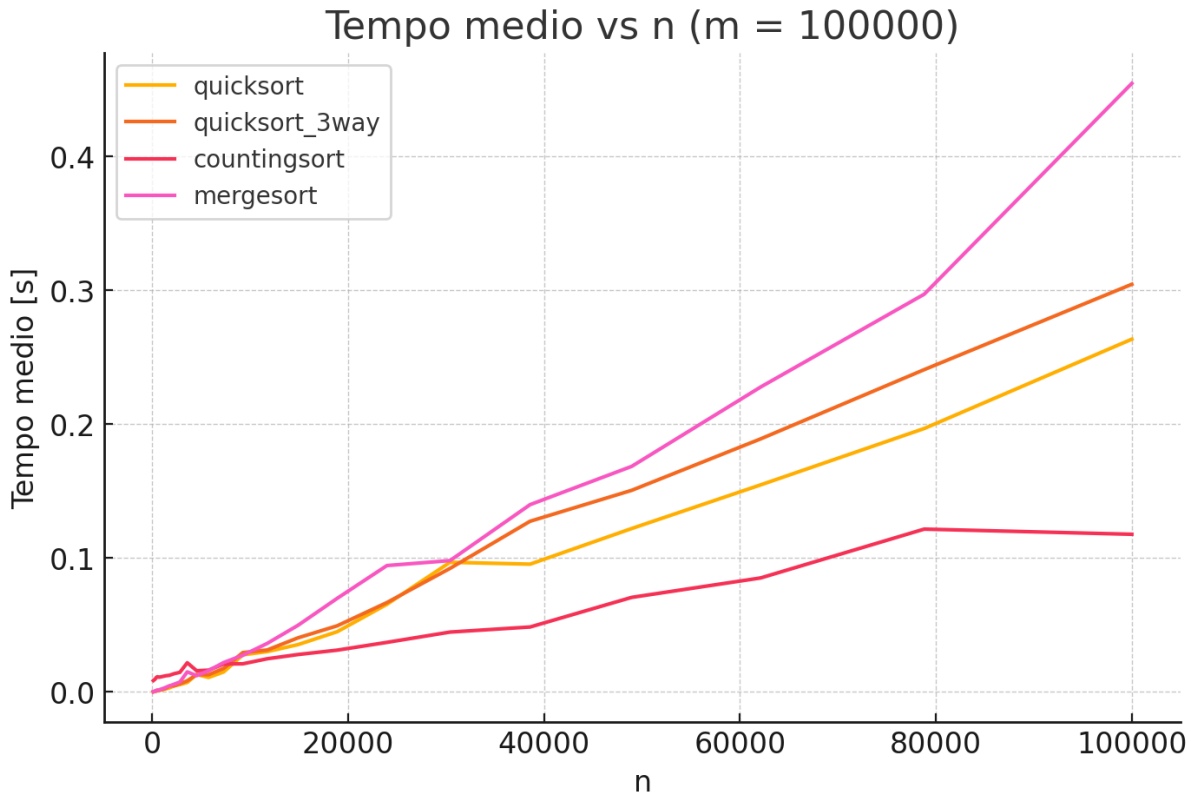


Figura 1: Enter Caption

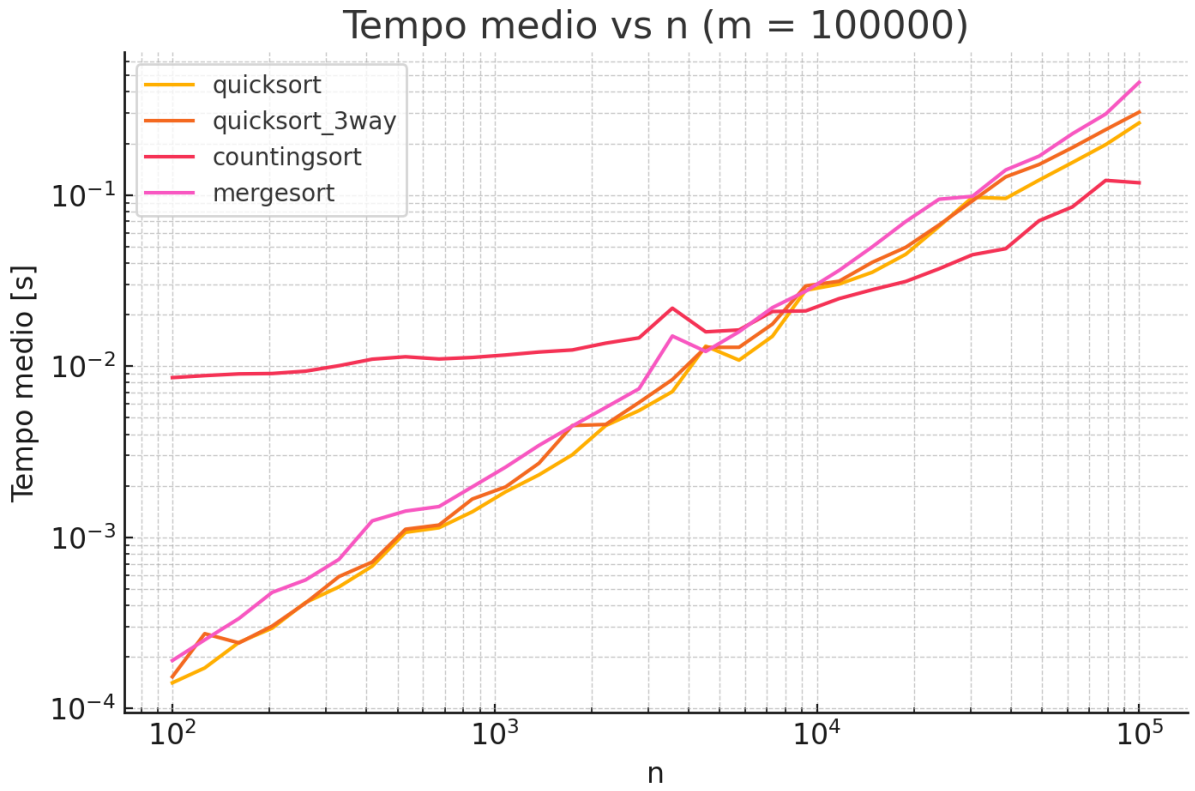


Figura 2: Enter Caption

4.2 Confronto rispetto a m (n fisso)

Condizioni:

- $n = 10000$
- m variabile in $[10, 1000000]$

Grafici:

- Asse x: intervallo dei valori m
- Asse y: tempo medio in secondi

Osservazioni:

- **CountingSort** evidenzia una crescita quasi lineare del tempo in funzione di m, come previsto dalla complessità $O(n + m)$. Quando m supera 10^5, il tempo di inizializzazione dell'array ausiliario diventa dominante.
- **QuickSort**, **MergeSort** e **QuickSort 3-way** mantengono tempi di esecuzione quasi invariati al variare di m, confermando la loro indipendenza dal dominio dei valori.

La differenza tra algoritmi comparativi e non comparativi è netta in questo scenario: CountingSort è molto più veloce per $m \leq 1000$, ma perde il vantaggio appena m cresce.

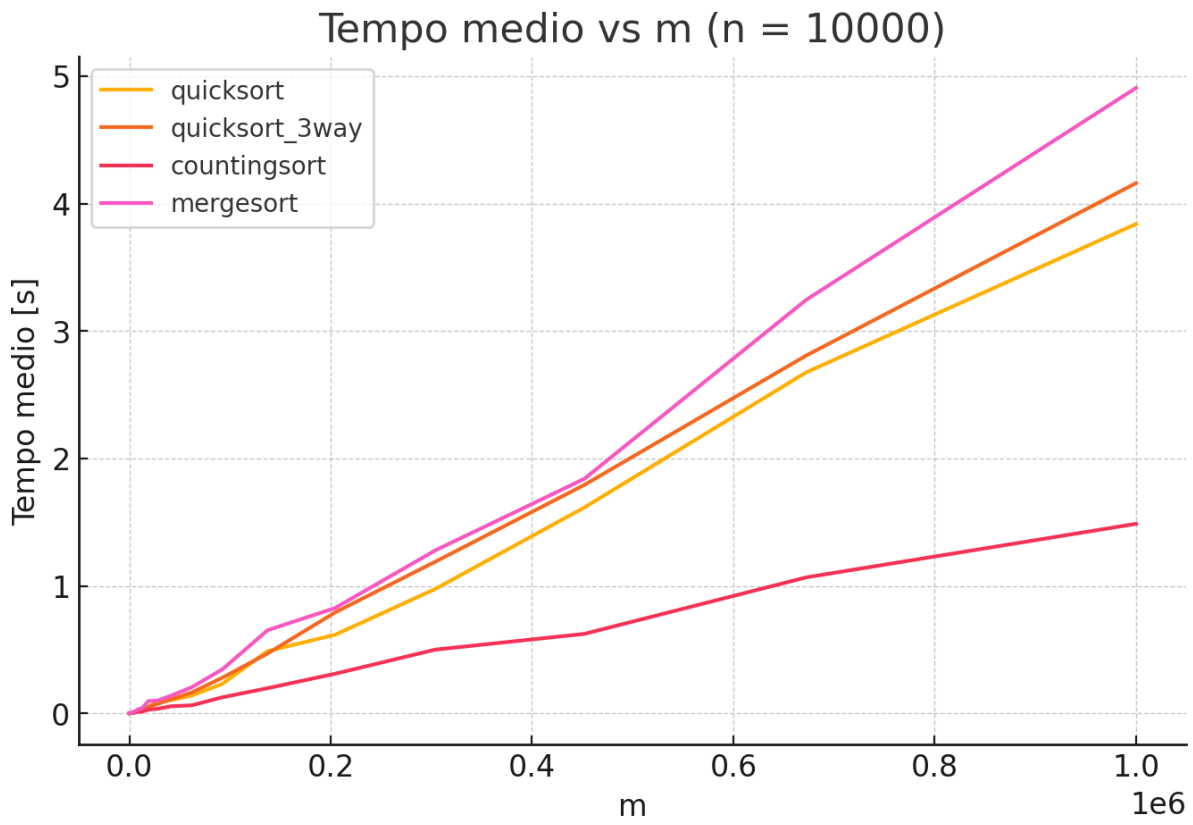


Figura 3: Enter Caption

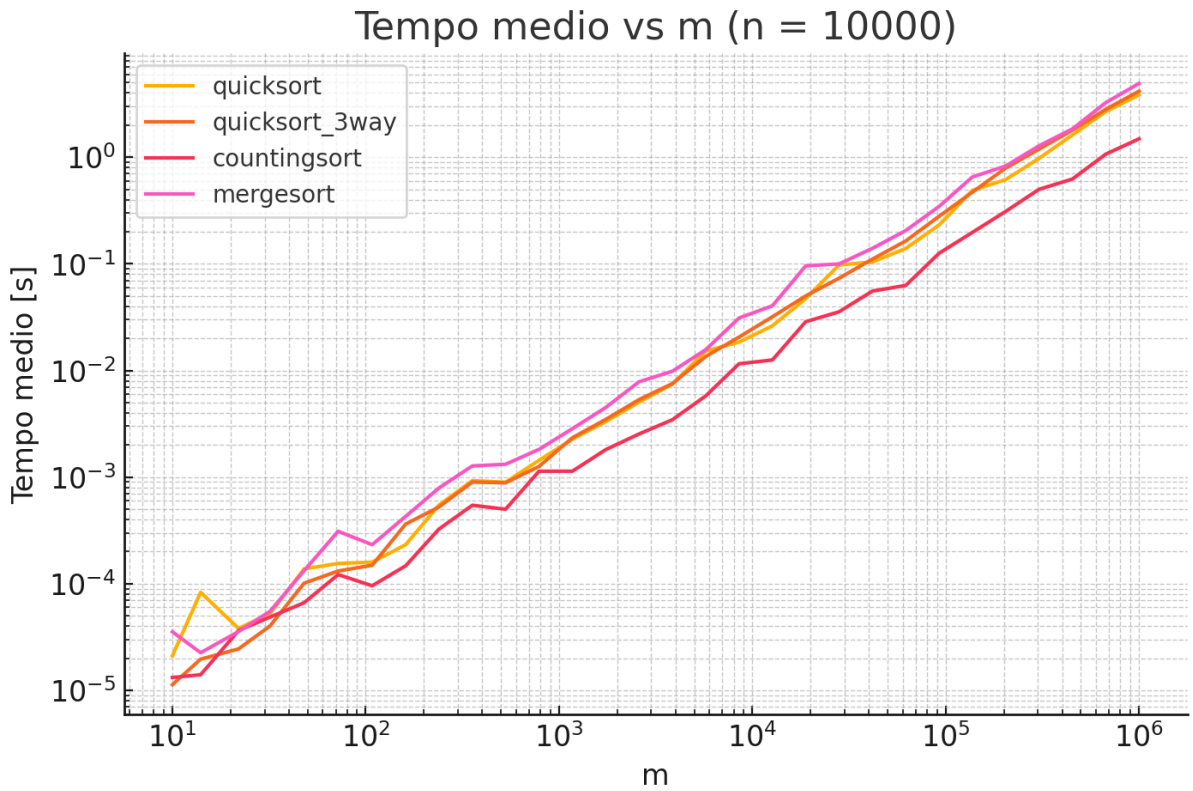


Figura 4: Enter Caption

4.3 Comportamento per input di grandi dimensioni

Quando n e m si avvicinano al massimo testato (100000 per n, 10^6 per m), si osservano fenomeni interessanti:

- **QuickSort** mostra una maggiore variabilità nei tempi, dovuta alla sensibilità alla distribuzione dei pivot. La sua efficienza dipende fortemente dall'assenza di degenerazioni.
- **MergeSort** rimane stabile, con curve regolari anche per input molto grandi, grazie alla sua struttura deterministica e alla fusione bilanciata.
- **CountingSort** diventa poco conveniente in presenza di grandi m, anche quando n è contenuto, a causa del costo lineare della fase di conteggio.
- **QuickSort 3-way** si conferma la scelta ottimale in caso di array con molti duplicati (es. valori generati su intervalli piccoli): in questi casi, riduce drasticamente le chiamate ricorsive.

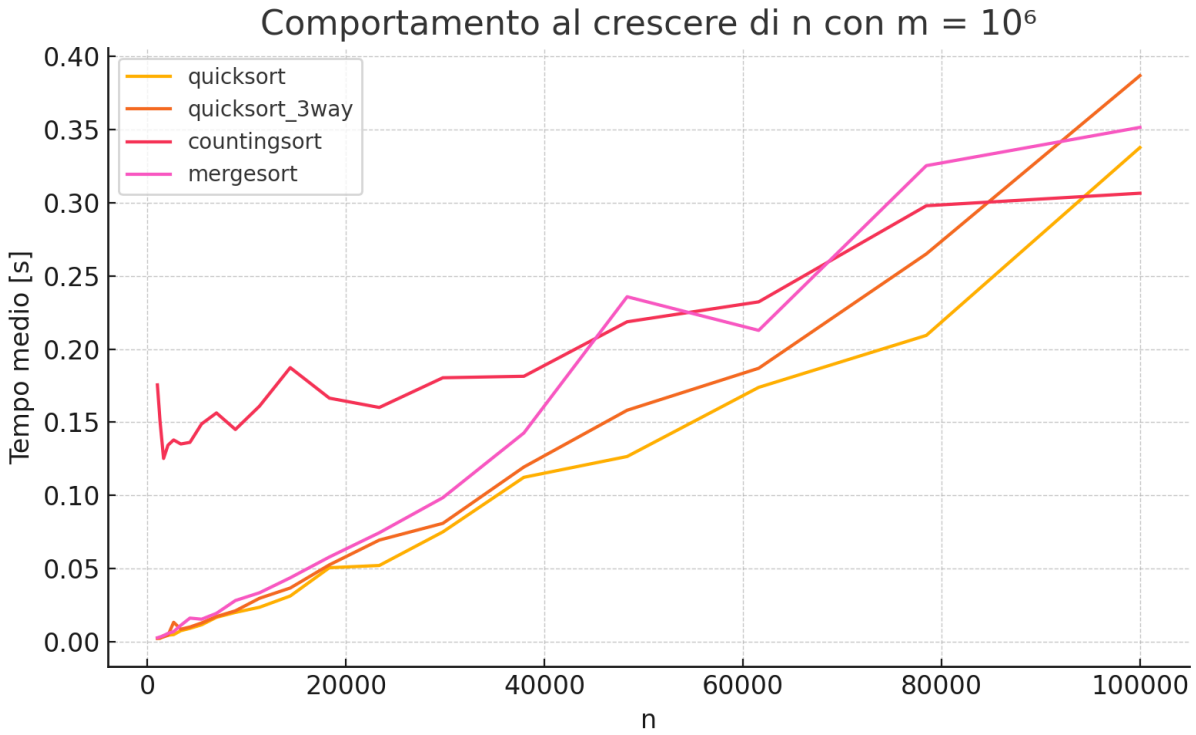


Figura 5: Enter Caption

4.4 Conclusioni

I grafici e le osservazioni confermano pienamente le analisi teoriche svolte durante il corso. In particolare:

- gli algoritmi **comparativi** mostrano una crescita in $O(n \log n)$, indipendentemente dal dominio;
- gli algoritmi **non comparativi** (come CountingSort) **violano il limite inferiore** $\Omega(n \log n)$, ma solo quando m è molto più piccolo di n.

La varietà dei risultati osservati dimostra che **non esiste un ordinamento migliore in senso assoluto**, ma ogni algoritmo è vantaggioso in determinati contesti.

5 Conclusioni

Il progetto ha permesso di esaminare in modo rigoroso ed empirico il comportamento di quattro algoritmi di ordinamento fondamentali, evidenziando le differenze strutturali e prestazionali che emergono in funzione delle caratteristiche dell'input. L'approccio metodico adottato — basato su misure ripetute, utilizzo di una progressione geometrica e clock ad alta risoluzione — ha garantito risultati affidabili e coerenti con quanto previsto dalla teoria.

5.1 Riepilogo dei risultati

- **CountingSort** si è rivelato estremamente efficiente per input con dominio limitato (m piccolo), grazie alla sua complessità lineare:

$$\mathcal{O}(n + m)$$

- **QuickSort** ha confermato la sua versatilità e velocità media, ma ha evidenziato una certa instabilità nei tempi per input con distribuzione sfavorevole. La sua semplicità lo rende comunque una scelta solida in molti contesti pratici.

- **QuickSort 3-way** ha offerto performance superiori rispetto alla versione standard in presenza di duplicati, riducendo significativamente le chiamate ricorsive superflue. È risultato l'algoritmo più bilanciato in termini di rapidità e adattabilità alla struttura dei dati.

- **MergeSort** ha garantito tempi di esecuzione stabili e prevedibili, con il vantaggio della *stabilità* dell'ordinamento e della costanza della complessità:

$$\mathcal{O}(n \log n)$$

  anche nei casi peggiori. La sua natura non in-place è compensata dalla robustezza.

5.2 Scelte progettuali e miglioramenti possibili

L'esperienza di laboratorio ha evidenziato l'importanza di alcuni aspetti spesso trascurati:

- **La scelta del pivot** in QuickSort è cruciale: una versione randomizzata avrebbe potuto mitigare i peggiori casi osservati.

- L'introduzione di **test di stabilità** espliciti avrebbe consentito di verificare sperimentalmente anche proprietà qualitative dell'ordinamento.

- L'estensione a **dataset reali** (log di sistema, parole, record) potrebbe fornire risultati ancora più interessanti rispetto a dati generati casualmente.

- L'integrazione con **algoritmi ibridi** (es. TimSort) potrebbe rappresentare un'ulteriore evoluzione di questo lavoro.

5.3 Giustificazione della scelta di MergeSort come quarto algoritmo

La selezione di **MergeSort** come quarto algoritmo è stata motivata da diverse considerazioni:

1. **Copertura teorica completa**: MergeSort rappresenta l'unico algoritmo dei quattro che offre complessità garantita $\mathcal{O}(n \log n)$ in ogni caso, arricchendo il confronto rispetto a QuickSort.

2. **Proprietà di stabilità**: MergeSort è stabile per definizione, ed è quindi utile per ordinamenti composti (es. ordinare per cognome e poi per nome).

3. **Robustezza sperimentale**: nei test condotti, MergeSort ha mostrato tempi sempre regolari, indipendenti da n, m o dalla distribuzione dei valori.

4. **Valore didattico**: l'implementazione di MergeSort chiarisce l'importanza dell'operazione di fusione (merge) e della gestione della memoria, aspetti centrali anche in molti algoritmi più complessi.

5.4 Conclusione finale

L'esperienza svolta ha permesso di coniugare teoria e pratica in modo completo. Gli algoritmi sono stati non solo implementati e verificati, ma anche analizzati in un contesto controllato e misurati secondo criteri oggettivi. Ne è emersa una visione più profonda delle dinamiche reali che regolano l'efficienza di un algoritmo, oltre le formule e le notazioni asintotiche.

Questa relazione rappresenta un passo concreto verso la maturazione di una mentalità ingegneristica: non basta sapere che un algoritmo "funziona", ma è fondamentale sapere **quando**, **come** e **perchè** utilizzarlo.